

How to Actually Write C

How to actually write non-trivial programs in C

Table of Contents

1. [Introduction](#)
2. [How to actually: do manual memory management](#)
 1. [Grouping allocations](#)
 2. [Use the stack](#)
 3. [TLDR](#)
3. [How to actually: do polymorphic data](#)
 1. [TLDR](#)
4. [How to actually: simulate object.method\(\) / organize data and logic](#)
 1. [TLDR](#)
5. [How to actually: do generics](#)
 1. [Non-intrusive data structures + macros](#)
 2. [Function pointers](#)
 3. [_Generic](#)
 4. [TLDR](#)
6. [Footnotes](#)

Introduction

I've found that most learning material on C teach you the *basics* of the language (what are pointers, how to call `malloc()` and `free()`, etc.), but they don't actually teach you how to write non-trivial and non-pedagogical programs in C.

I've recently been playing with C¹ and have been learning out how to *actually* write C programs, so I'll share some tips here. A lot of these learnings are not taught in university courses or books, or are at least briefly mentioned, while I think they are super important.

So if you're trying out C for fun, to learn, or you need to work with some C library,

hopefully this guide will help.

I'll keep updating this post as I learn new things.

How to actually: do manual memory management

Grouping allocations

I've always wondered how C coders actually do manual memory management. It felt like it was so difficult to not drown in the complexity of managing the lifetimes of each memory allocation, to make sure each `malloc()` is `free()`'d

But we're programmers. And the way we reduce the complexity of difficult problems is through abstraction.

So instead of managing the lifetime of individual memory allocations, let's abstract that, and manage a *group* of memory allocations.

It sounds simple, but depending on how you group allocations, you can literally reduce the number of `malloc()`'s and `free()`'s from thousands to a handful.

The most useful strategy is *arena allocation*. In this technique, an *arena* is a group of allocations that all share a similar *end* to their lifetime. This means you only need to free the memory once.

The most compelling example is an arena used for a single frame in a video game. In rendering systems, we typically do a bunch of work and allocate a bunch of memory to show graphics on the screen. And a lot of the time, the memory is unneeded after that frame is shown. All these allocations "die" at the end of the frame. So we can group them into a single buffer (arena), and free that buffer at the end of the frame (though usually the buffer is not freed but reset so it can be used for the next frame).

Here's an example:

```
#include "arena.h"

void process_frame(Arena *arena) {
    // allocate some game objects
    Cat *cat = (Cat *) arena_alloc(sizeof(Cat));
    Dog *dog = (Dog *) arena_alloc(sizeof(Dog));
    Frog *frog = (Frog *) arena_alloc(sizeof(Frog));
```

```

Cow *cow = (Cow *) arena_alloc(sizeof(Cow));

// This function could potentially allocate thousands of objects,
// but we're not stressed! We just clear everything at the end, without a
// care in the world!
do_some_other_stuff(arena);

// Reset arena back to initial state, but don't free the memory so we can
// use it for next frame
arena_clear();
}

```

They key takeaway from the example is that we could allocate a lot of data, but we only need to call one function (`arena_clear()`) to clean it up.

Arenas and “grouping” allocations isn’t just something that alleviates complexity, it can also lead to performance optimizations. For example, allocating heap memory is relatively slow, due to the bookkeeping the OS needs to do. When you initialize an Arena, it can *preallocate* memory from the OS (typically a page, so 4kb), meaning all that expensive bookkeeping is done *once* at initialization.

Combine that with the fact that most arenas are *linear allocators*, which literally just have to increment a pointer to allocate.

This makes all the `arena_alloc()` calls above *extremely* fast.

A pretty good resource on arenas is Ryan Fleury’s [article](#) on the subject. For implementations, I would look at [gingerBill’s](#) and the one from the [PBR book](#).

Memory allocation strategies in general are a very fun rabbit hole to go down if you’re interested in performance. I recommend gingerBill’s [series on memory allocation strategies](#).

Use the stack

One secret trick no one tells you is that the OS gives you a memory manager for free, the stack. Additionally, the stack is much faster than the heap.

```

typedef struct {
    uint16_t health;
    Vector2 pos;
} Alien;

```

```

#define ALIEN_MAX_COUNT 25

typedef struct {
    Alien aliens[ALIEN_MAX_COUNT];
    uint8_t alien_count;
} GameState;

void game_state_add_alien(GameState *gs, Alien alien) {
    if (gs->alien_count == ALIEN_MAX_COUNT) return;
    gs->aliens[gs->alien_count++] = alien;
}

int main() {
    // `game_state.aliens` is on the stack
    GameState game_state;
    game_state.alien_count = 0;

    game_state_add_alien(&game_state, (Alien){ .health = 100, .pos = { 0.0, 0.0 }});
}

```

TLDR

1. Learn to group allocations (reduces the amount of times you have to call `free()` and can improve performance)
2. Avoid dynamic memory allocation by using the stack

How to actually: do polymorphic data

Often, we have data that can take up multiple forms. The way most people learn how to implement this is with OOP and classes. The better way, in my opinion, is through **tagged unions**.

You've probably been exposed to tagged unions before, maybe Rust's enums, or Typescript's [discriminated unions](#).

In Rust:

```

enum Shape {
    Circle { radius: i32 },
    Square { side: i32 },
    Rect { width: i32, height: i32 }
}

```

```
fn print_shape(shape: Shape) {
    match shape {
        Shape::Circle { radius } => println!("Circle with radius: {}", radius),
        Shape::Square { side } => println!("Square with side len: {}", square),
        Shape::Rect { width, height } => println!("Rect {} x {}", width, height),
    }
}
```

In Typescript:

```
type Shape = { type: 'circle', radius: number } | { type: 'square', side: number } | { type: 'rect', width: number, height: number }

function printShape(shape: Shape) {
    switch (shape.type) {
        case 'circle':
            console.log('Circle with radius:', shape.radius)
            break;
        case 'square':
            console.log('Square with side len:', shape.side)
            break;
        case 'rect':
            console.log(`Rect ${shape.width}x${shape.height}`)
            break;
    }
}
```

Both of these can be considered *tagged unions*. In Rust, the *tag* is implicit. The compiler will usually pick an unsigned integer to denote the tag and pattern matching checks it for you. In Typescript, you have explicitly write the tag and check it yourself.

This is the best way to represent polymorphic data for a discrete amount of values, and we can do it in C too!

```
typedef enum {
    ShapeTagCircle,
    ShapeTagSquare,
    ShapeTagRect
} ShapeTag;

typedef union {
    ShapeTag tag;

    struct {
        int radius;
    } circle;

    struct {
        int side;
    } square;
}
```

```

    } square;

    struct {
        int w, h;
    } rect;
} Shape;

void print_shape(Shape shape) {
    switch (shape.tag) {
        case ShapeTagCircle:
            printf("Circle with radius: %d\n", shape.circle.radius);
            break;
        case ShapeTagSquare:
            printf("Square with side len: %d\n", shape.square.side);
            break;
        case ShapeTagRect:
            printf("Rect: %dx%d\n", shape.rect.w, shape.rect.h);
            break;
    }
}

```

In C, each field of a union is one of the shapes the data can take. Very similar to our Rust and Typescript examples above. However, a `union` doesn't have a tag by default, so we define one and set it. Now, we *could* do the same without the tag:

```

typedef union {
    struct {
        int radius;
    } circle;

    struct {
        int side;
    } square;

    struct {
        int w, h;
    } rect;
} Shape;

```

But we wouldn't be able to tell *which* shape it is at *runtime*. But if you are writing a program and you precisely know which specific variant each `Shape` is, you can save some space by omitting the tag.

TLDR

Use tagged unions

How to actually: simulate `object.method()` / organize data and logic

What most people love about OOP is actually getting to call methods using the dot syntax: `object.method()`. Not only is it shorter, but it allows you to easily organize functions associated with objects.

In C you don't get this, but it is still useful to associate functions with the data they operate on. You can *approximate* this by simply prefixing the function name with the object name: `object_method(&obj)`

```
typedef struct {
    const char *name;
    int health;
    int lives;
} Cat;

Cat cat_init(const char *name) {
    return (Cat){
        .name = name,
        .health = 100,
        .lives = 9,
    };
}

const char *cat_print_name(const Cat *cat) {
    printf("Name: %s\n", cat->name);
}

void test_cat(Cat *cat) {
    cat_print_name(cat);
}
```

This lets you associate data and logic.

If you come from languages like Go, Rust, or Zig, they already have syntax sugar to do this for you. For example, in Go:

```
type Cat struct {
    Name    stringj
    Health  int
    Lives   int
}

func (cat *Cat) PrintName() {
    fmt.Printf("Name: %s\n", cat.Name)
}

func testCat(cat *Cat) {
    cat.PrintName()
}
```

In Rust:

```
struct Cat {
    name: String,
    health: i32,
    lives: i32,
}

impl Cat {
    fn print_name(&self) {
        println!("Name: {}", self.name);
    }
}

fn test_cat(cat: &Cat) {
    cat.print_name();
}
```

And in Zig:

```
const Cat = struct {
```

```

    name: []const u8,
    health: i32,
    lives: i32,

    fn print_name(self: *Cat) void {
        std.debug.print("Name: {s}\n", .{self.name});
    }
};

fn test_cat(cat: *Cat) void {
    cat.print_name();
}

```

It would be nice to have `cat.print_name()` in C, but I have found that once you get used to this syntax it's not that bad.

TLDR

You can organize data and logic better by prefixing functions that are associated with data by the name of the data type: `cat_meow(&cat)`

How to actually: do generics

A lot of the times a generic data structure might only need a handful of variants. If that's the case, use a tagged union.

If the amount of variants is too unwieldy, then you'll need a different strategy.

Non-intrusive data structures + macros

Let's say you're building a dynamically resizable array data structure. In a program you will likely have many array types, so creating a tagged union simply isn't feasible.

One pattern is *non-intrusive containers*. This basically means the container doesn't know anything about the underlying type it stores, and you give it that information when you want to manipulate it.

Let's start with an example of making array:

```
typedef struct {
```

```

    char *ptr;
    size_t len;
    size_t cap;
} Array;

void _array_init(Array *arr, size_t cap, size_t element_size) {
    arr->len = 0;
    arr->cap = cap;
    arr->ptr = (char *)malloc(cap * element_size);
}

void _array_grow(Array *arr, size_t new_cap, size_t element_size) {
    assert(new_cap >= arr->cap);
    arr->ptr = (char *)realloc(arr->ptr, new_cap * element_size);
    arr->cap = new_cap;
}

void *_array_pop(Array *arr, size_t element_size) {
    return (void *)&arr->ptr[--arr->len * element_size];
}

```

When you want to initialize, grow, or pop from the `Array`, we give it information pertaining to its underlying type. In this case, that's the size of the element bytes (`sizeof(T)`)

Now, it's annoying to pass in the size everywhere, so we can create some helper macros:

```

#define array_type(T) \
    struct { \
        T *ptr; \
        size_t len; \
        size_t cap; \
    }

#define array_init(T, a, cap) _array_init((Array *)(a), (cap), sizeof(T))
#define array_grow(T, a, cap) _array_grow((Array *)(a), (cap), sizeof(T))

#define array_pop(T, a) (T *)(_array_pop((Array *)(a), sizeof(T)))
#define array_push(a, value) (a)->ptr[(a)->len++] = (value)

typedef array_type(float ) FloatArray;

void example() {
    FloatArray arr;
    array_init(float , &arr, 8);
}

```

```

    array_push(float , &arr, 69);
    float val = array_pop(float , &arr);
}

```

The great part about this is that the compiler should be able to optimize out the passing of the size at runtime, since the sizes are known at compile time.

Function pointers

Sometimes, the information we want to pass to a generic container or data structure is *logic*, and we can do this with function pointers.

Let's say I want to implement a generic binary search function on the array type we defined earlier:

```

typedef enum {
    OrderLess,
    OrderEqual,
    OrderGreater
} Order;

// Comparison function, returns:
// - a value > 0 if the second argument is greater than the first
// - a value < 0 if the second argument is less than the first
// - a value == 0 if they are both equal
typedef Order (*ComparisonFn)(void *, void *);

int _array_binary_search(Array *arr, size_t elem_size, void *search_elem,
                        ComparisonFn cmp) {
    size_t h = arr->len;
    size_t l = 0;

    while (l < h) {
        size_t m = l + (h - l) / 2;
        void *a = (void *)&arr->ptr[m * elem_size];
        Order cmp_result = cmp(search_elem, a);
        if (cmp_result == OrderEqual)
            return m;

        if (cmp_result == OrderGreater) {
            l = m + 1;
        } else {
            h = m;
        }
    }
}

```

```

}

return -1;
}

```

I can then define another macro to make this a little easier to call:

```

#define array_binary_search(T, a, search, cmp) \
    _array_binary_search((Array *)(a), sizeof(T), (search), (ComparisonFn)(cmp))

```

Now let's say we're making a text editor and we have a `TextPos` struct that contains a line and a column for a text position:

```

typedef struct {
    int col;
    int row;
} TextPos;

typedef array_type(TextPos) TextPosArray;

// Comparison function for TextPos
Order textpos_cmp(TextPos *a, TextPos *b) {
    if (a->row < b->row)
        return OrderLess;
    if (a->row > b->row)
        return OrderGreater;
    if (a->col < b->col)
        return OrderLess;
    if (a->col > b->col)
        return OrderGreater;
    return OrderEqual;
}

```

Now we can use `TextPos` and its comparison function like so:

```

int main(void) {

```

```

TextPosArray arr;
array_init(TextPos, &arr, 10);
array_push(&arr, ((TextPos){.col = 1, .row = 0}));
array_push(&arr, ((TextPos){.col = 10, .row = 0}));
array_push(&arr, ((TextPos){.col = 1, .row = 1}));

TextPos search = {.col = 1, .row = 1};
int idx = array_binary_search(TextPos, &arr, &search, textpos_cmp);

assert(idx == 2);
return 0;
}

```

One thing to note is that calling a function pointer has some performance drawbacks, but can be avoided if the compiler can inline the function call. Though, sometimes, this performance hit is negligible. For example, the C stdlib uses function pointers for some I/O stuff, where the bottleneck will be doing I/O, not function pointers.

_Generic

So far, the way we've been doing generics is by essentially passing information or logic to functions. `_Generic()` is the reverse of that. An example:

```

Order int_compare(int *a, int *b) {
    if (a > b)
        return OrderGreater;
    if (a < b)
        return OrderLess;
    return OrderEqual;
}

// our function from earlier
Order textpos_cmp(TextPos *a, TextPos *b);

#define compare(X, Y) \
    _Generic((X), int *: int_compare, TextPos *: textpos_cmp)(X, Y)

void test_generic_compare(TextPos *textpos1, TextPos *textpos2, int *int1, int *int2) {
    Order result = compare(textpos1, textpos2);
    Order result2 = compare(int1, int2);
}

```

1. *Non-intrusive data structures*: you can get around not having generics by passing in type-specific information as parameters
2. *Function pointers*: A way to inject logic, make sure its inlinable if performance matters
3. *_Generic*: When paired with a wrapper macro, can let you select which function to call at compile-time

Footnotes

1↩

I'm a long time fan and user of complex languages like Rust and Haskell, so I became intrigued when I noticed Rasmus Andersson, who writes a lot of cool stuff (almost all in C), talked about how C's simplicity allowed him to focus on data and logic, instead of being distracted by needless "abstraction building exercises".

This made me want to try C out for myself. I had experience with C from 1 year of uni before I dropped out, and from dealing with FFmpeg's source.